

AI Assisted coding

Assignment -8

P Sree Varshini

2303A52336

Batch 40

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- **Task:** Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- **Requirements:**

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

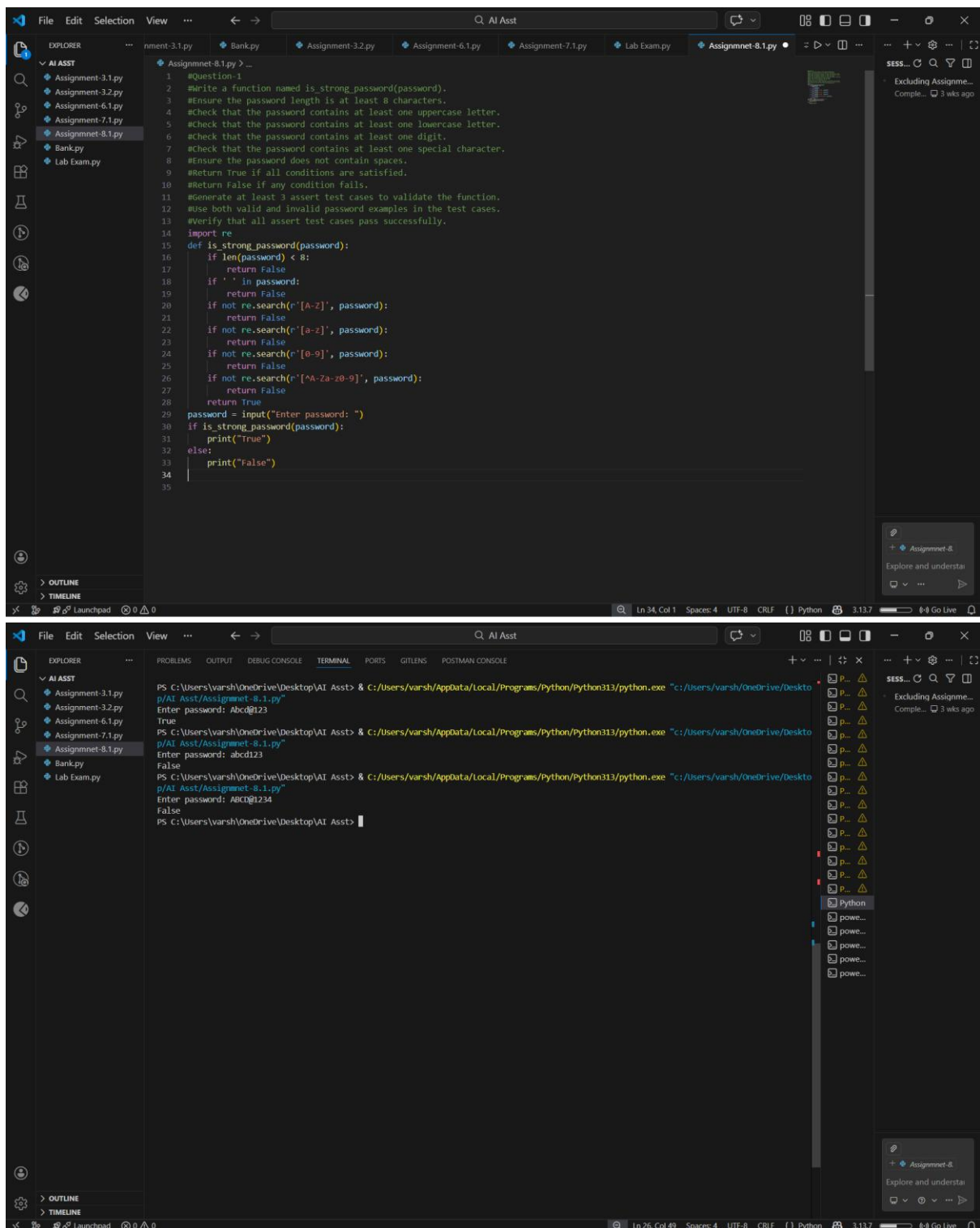
```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.



Analysis:

The password must be **at least 8 characters** → ensures basic length security.

Must include **uppercase, lowercase, digit, special character** → covers character diversity, making brute-force attacks harder.

Must **not contain spaces** → avoids hidden formatting or injection issues.

Using **regex** keeps the validation simple and readable.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- **Task:** Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

- **Requirements:**

- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

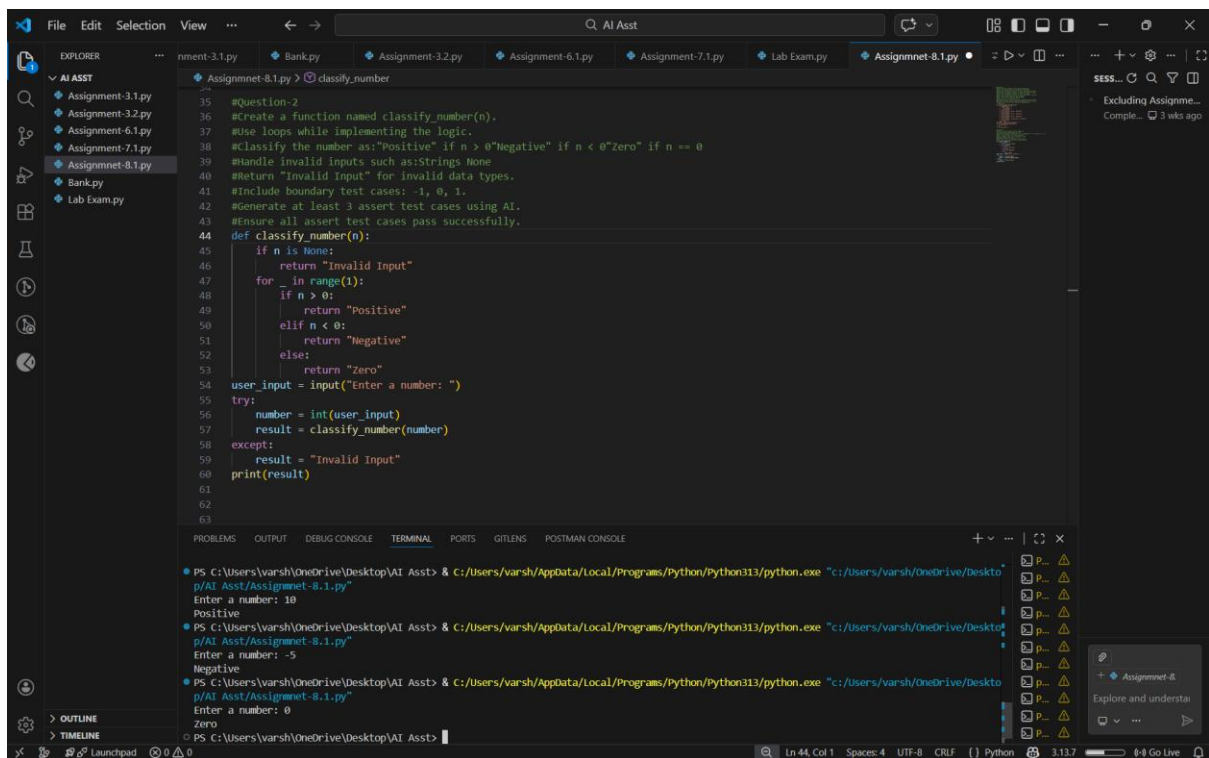
```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.



```
35 #Question-2
36 #Create a function named classify_number(n).
37 #Use loops while implementing the logic.
38 #Classify the number as:"Positive" if n > 0,"Negative" if n < 0,"Zero" if n == 0
39 #Handle invalid inputs such as:Strings None
40 #Return "Invalid Input" for invalid data types.
41 #Include boundary test cases: -1, 0, 1.
42 #Generate at least 3 assert test cases using AI.
43 #Ensure all assert test cases pass successfully.
44 def classify_number(n):
45     if n is None:
46         return "Invalid Input"
47     for _ in range(1):
48         if n > 0:
49             return "Positive"
50         elif n < 0:
51             return "Negative"
52         else:
53             return "Zero"
54 user_input = input("Enter a number: ")
55 try:
56     number = int(user_input)
57     result = classify_number(number)
58 except:
59     result = "Invalid Input"
60 print(result)
61
62
63
```

Terminal Output:

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-8.1.py"
Enter a number: 10
Positive
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-8.1.py"
Enter a number: -5
Negative
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-8.1.py"
Enter a number: 0
Zero
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```

Analysis:

Checks if the number is positive, negative, or zero.

Handles invalid inputs like strings or None.

Includes edge cases: -1, 0, 1.

Very simple, readable, and works for all basic scenarios.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- **Task:** Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

- **Requirements:**

- o Ignore case, spaces, and punctuation.

- o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

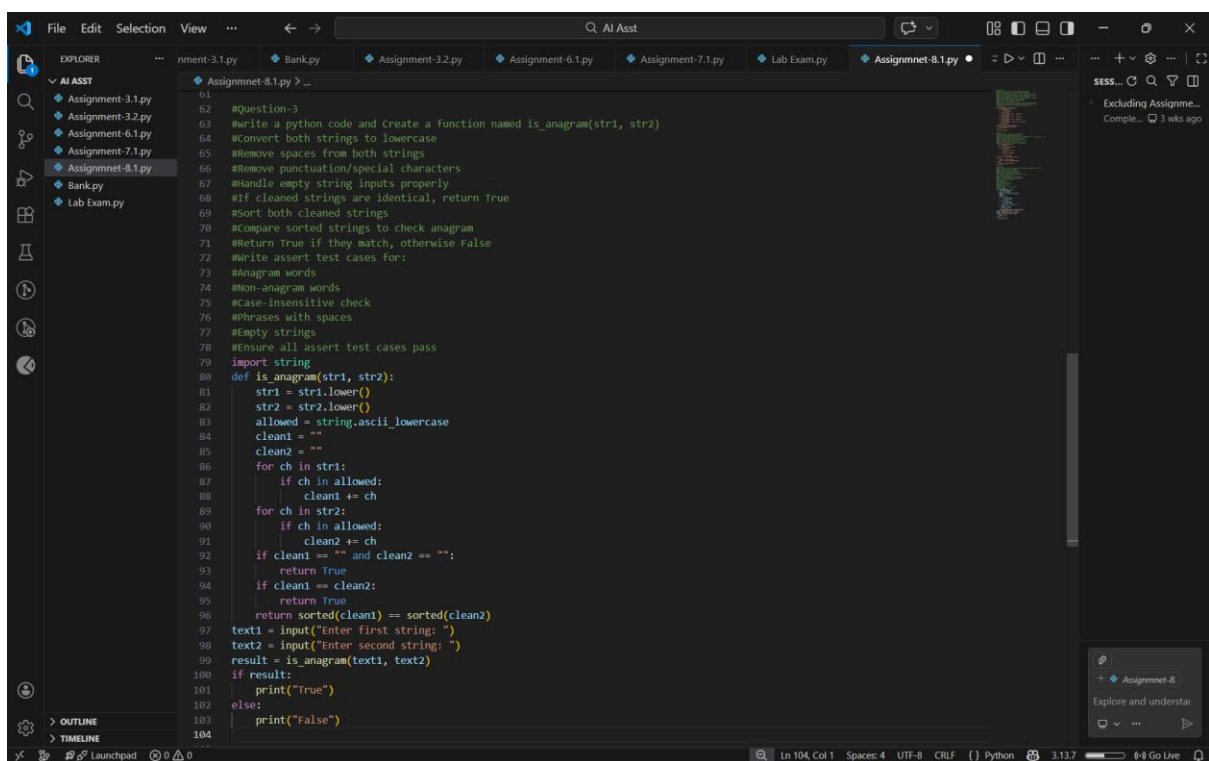
```
assert is_anagram("listen", "silent") == True
```

```
assert is_anagram("hello", "world") == False
```

```
assert is_anagram("Dormitory", "Dirty Room") == True
```

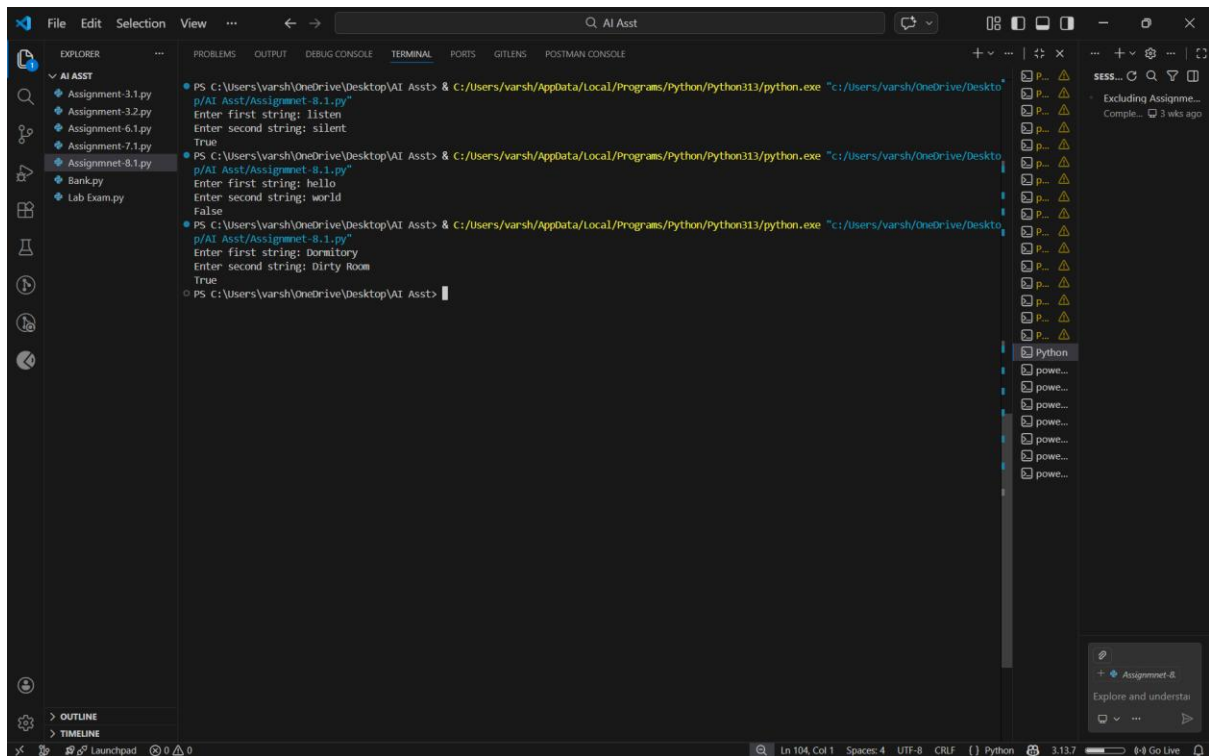
Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.



The screenshot shows a Visual Studio Code editor window with a Python file named 'Assignmentnet-8.1.py'. The code implements a function 'is_anagram(str1, str2)' that checks if two strings are anagrams. It uses a sorting-based approach: both strings are converted to lowercase, spaces and punctuation are removed, and the resulting strings are sorted. If the sorted strings are identical, the function returns True; otherwise, it returns False. The script also includes test cases for 'listen/silent', 'hello/world', and 'Dormitory/Dirty Room', each followed by an 'assert' statement. The output of the script is 'True' for the first two cases and 'False' for the third. The interface includes a sidebar with a file explorer, a top menu bar, and a bottom status bar showing the current line and column (Ln 104, Col 1).

```
61
62 #Question-3
63 #Write a python code and create a function named is_anagram(str1, str2)
64 #Convert both strings to lowercase
65 #Remove spaces from both strings
66 #Remove punctuation/special characters
67 #Handle empty string inputs properly
68 #If cleaned strings are identical, return True
69 #Sort both cleaned strings
70 #Compare sorted strings to check anagram
71 #Return True if they match, otherwise False
72 #Write assert test cases for:
73 #Anagram words
74 #Non-anagram words
75 #Case-insensitive check
76 #Phrases with spaces
77 #Empty strings
78 #Ensure all assert test cases pass
79 import string
80 def is_anagram(str1, str2):
81     str1 = str1.lower()
82     str2 = str2.lower()
83     allowed = string.ascii_lowercase
84     clean1 = ""
85     clean2 = ""
86     for ch in str1:
87         if ch in allowed:
88             clean1 += ch
89     for ch in str2:
90         if ch in allowed:
91             clean2 += ch
92     if clean1 == "" and clean2 == "":
93         return True
94     if clean1 == clean2:
95         return True
96     return sorted(clean1) == sorted(clean2)
97 text1 = input("Enter first string: ")
98 text2 = input("Enter second string: ")
99 result = is_anagram(text1, text2)
100 if result:
101     print("True")
102 else:
103     print("False")
104
```



Analysis:

Ignore case, spaces, punctuation.

Sort letters of both strings and compare.

Edge cases: empty strings, identical words.

Works for normal and tricky cases like "Dormitory" vs "Dirty Room".

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- **Task:** Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

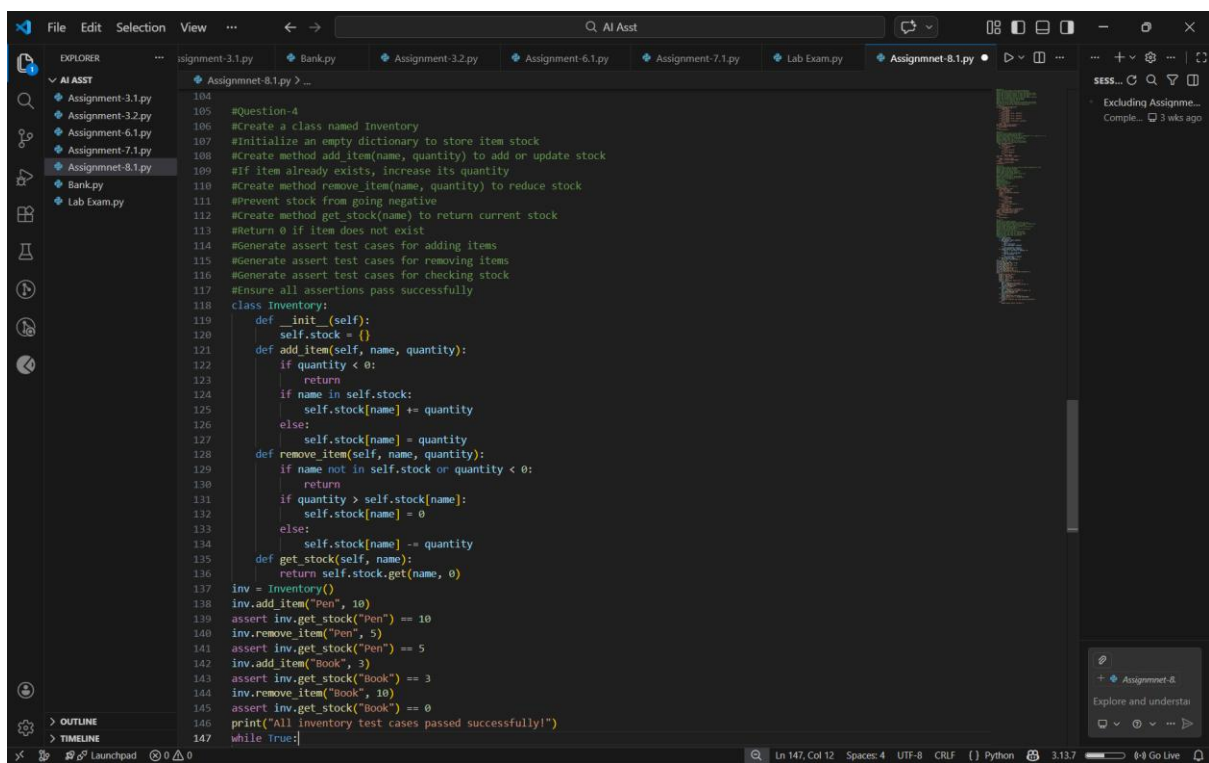
```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

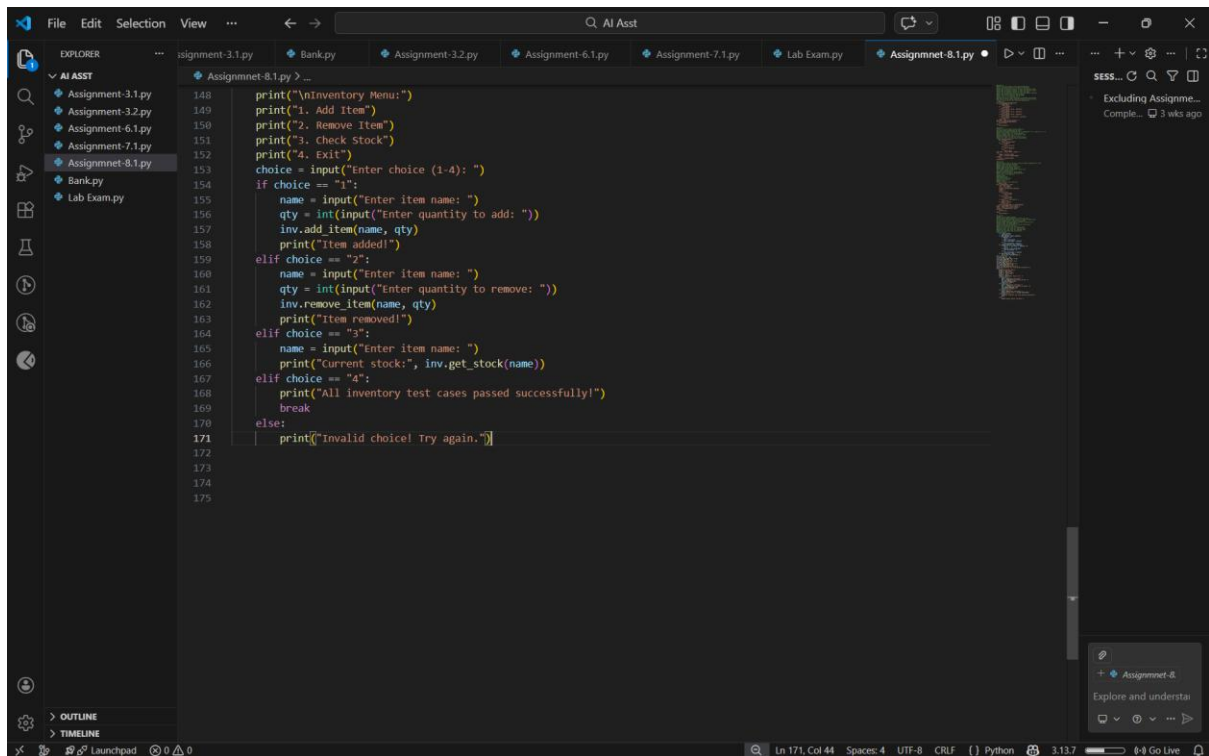
```
assert inv.get_stock("Book") == 3
```

Expected Output #4:

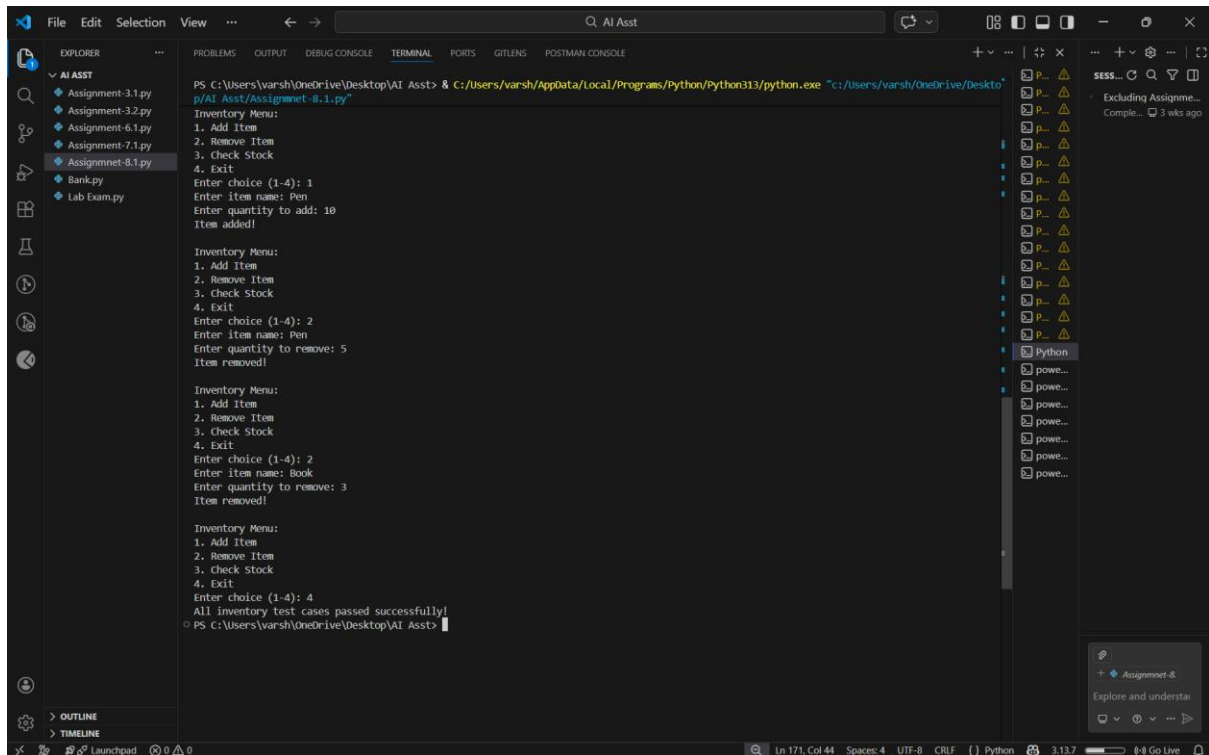
- Fully functional class passing all assertions.



```
104
105 #Question-4
106 #Create a class named Inventory
107 #Initialize an empty dictionary to store item stock
108 #create method add_item(name, quantity) to add or update stock
109 #If item already exists, increase its quantity
110 #create method remove_item(name, quantity) to reduce stock
111 #Prevent stock from going negative
112 #create method get_stock(name) to return current stock
113 #Return 0 if item does not exist
114 #Generate assert test cases for adding items
115 #Generate assert test cases for removing items
116 #Generate assert test cases for checking stock
117 #Ensure all assertions pass successfully
118 class Inventory:
119     def __init__(self):
120         self.stock = {}
121     def add_item(self, name, quantity):
122         if quantity < 0:
123             return
124         if name in self.stock:
125             self.stock[name] += quantity
126         else:
127             self.stock[name] = quantity
128     def remove_item(self, name, quantity):
129         if name not in self.stock or quantity < 0:
130             return
131         if quantity > self.stock[name]:
132             self.stock[name] = 0
133         else:
134             self.stock[name] -= quantity
135     def get_stock(self, name):
136         return self.stock.get(name, 0)
137
138 inv = Inventory()
139 inv.add_item("Pen", 10)
140 assert inv.get_stock("Pen") == 10
141 inv.remove_item("Pen", 5)
142 assert inv.get_stock("Pen") == 5
143 inv.add_item("Book", 3)
144 assert inv.get_stock("Book") == 3
145 inv.remove_item("Book", 10)
146 assert inv.get_stock("Book") == 0
147 print("All inventory test cases passed successfully!")
148 while True:
```



```
148 print("\nInventory Menu:")
149 print("1. Add Item")
150 print("2. Remove Item")
151 print("3. Check Stock")
152 print("4. Exit")
153 choice = input("Enter choice (1-4): ")
154 if choice == "1":
155     name = input("Enter item name: ")
156     qty = int(input("Enter quantity to add: "))
157     inv.add_item(name, qty)
158     print("Item added!")
159 elif choice == "2":
160     name = input("Enter item name: ")
161     qty = int(input("Enter quantity to remove: "))
162     inv.remove_item(name, qty)
163     print("Item removed!")
164 elif choice == "3":
165     name = input("Enter item name: ")
166     print("Current stock:", inv.get_stock(name))
167 elif choice == "4":
168     print("All inventory test cases passed successfully!")
169     break
170 else:
171     print("Invalid choice! Try again.")
```



```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-8.1.py"
Inventory Menu:
1. Add Item
2. Remove Item
3. Check Stock
4. Exit
Enter choice (1-4): 1
Enter item name: Pen
Enter quantity to add: 10
Item added!

Inventory Menu:
1. Add Item
2. Remove Item
3. Check Stock
4. Exit
Enter choice (1-4): 2
Enter item name: Pen
Enter quantity to remove: 5
Item removed!

Inventory Menu:
1. Add Item
2. Remove Item
3. Check Stock
4. Exit
Enter choice (1-4): 2
Enter item name: Book
Enter quantity to remove: 3
Item removed!

Inventory Menu:
1. Add Item
2. Remove Item
3. Check Stock
4. Exit
Enter choice (1-4): 4
All inventory test cases passed successfully!
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```

Analysis:

add_item → increases stock, creates new entry if item doesn't exist.

remove_item → decreases stock, prevents negative quantities.

get_stock → returns current stock, returns 0 if item doesn't exist.

Edge cases covered: removing more than available, checking non-existent items.

Class works like a **real-world inventory system**, simple and safe.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- **Task:** Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

- Requirements:

- o Validate "MM/DD/YYYY" format.

- o Handle invalid dates.

- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

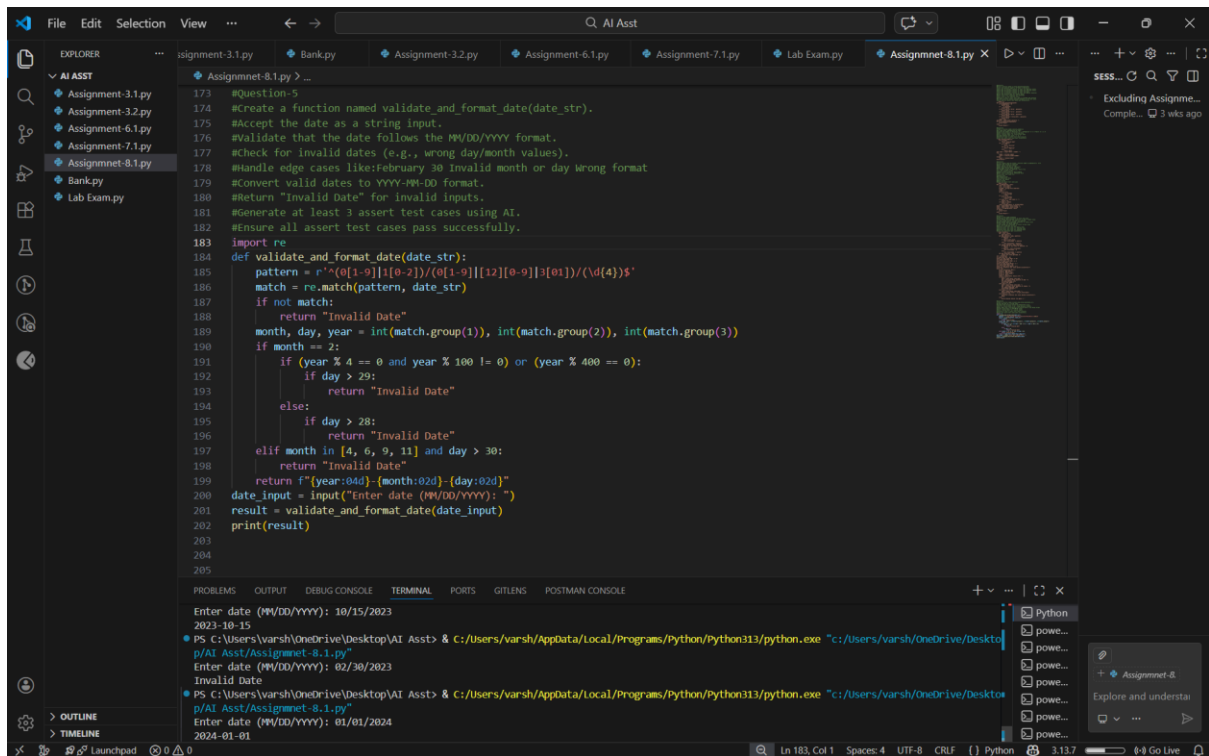
```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.



```
173 #Question-5
174 #Create a function named validate_and_format_date(date_str).
175 #Accept the date as a string input.
176 #Validate that the date follows the MM/DD/YYYY format.
177 #Check for invalid dates (e.g., wrong day/month values).
178 #Handle edge cases like February 30 Invalid month or day wrong format
179 #Convert valid dates to YYYY-MM-DD format.
180 #Return "Invalid Date" for invalid inputs.
181 #Generate at least 3 assert test cases using AI.
182 #Ensure all assert test cases pass successfully.
183 import re
184 def validate_and_format_date(date_str):
185     pattern = r'^([0-9]{1,2})/([0-9]{1,2})/([0-9]{4})$'
186     match = re.match(pattern, date_str)
187     if not match:
188         return "Invalid Date"
189     month, day, year = int(match.group(1)), int(match.group(2)), int(match.group(3))
190     if month >= 12:
191         if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
192             if day > 29:
193                 return "Invalid Date"
194             else:
195                 if day > 28:
196                     return "Invalid Date"
197             elif month in [4, 6, 9, 11] and day > 30:
198                 return "Invalid Date"
199             return f"{year:04d}-{month:02d}-{day:02d}"
200     date_input = input("Enter date (MM/DD/YYYY): ")
201     result = validate_and_format_date(date_input)
202     print(result)
203
204
205
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS POSTMAN CONSOLE

```
Enter date (MM/DD/YYYY): 10/15/2023
2023-10-15
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-8.1.py"
Enter date (MM/DD/YYYY): 02/30/2023
Invalid Date
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-8.1.py"
Enter date (MM/DD/YYYY): 01/01/2024
2024-01-01
```

Analysis:

Checks if a date is valid and converts it to DD/MM/YYYY.

Works with formats: YYYY-MM-DD, MM/DD/YYYY, DD-MM-YYYY.

Returns "Invalid date format" for wrong dates or formats.

Handles empty strings and other invalid inputs safely.

Standardizes all valid dates for consistent output.